Hari Patel

Problem 1

```
# (a) Finite difference formula for f''(x0)
print("-----------------------------------------------------");
print("(a) Finite difference formula for f''(x0):");
print("Expanded formula:");
print("f''(x0) ≈ [-f(x0 - h) + 4*f(x0 + h) - 3*f(x0 + 2*h) - f(x0)] / (2*h^2)");
# Define function and point
f := x → x^2*ln(x + 3) + cos(x)*exp(−3*x^2) :
x0 := 1 :
hlist := [0.1, 0.05, 0.01] :
# (b) Numerical approximations
f2 := diff(f(x), x$2) :
trueval := evalf(subs(x=x0, f2));
print("-----------------------------------------------------");
print("(b) Numerical approximations for f''(1):");
print(cat("True f''(1) = ", trueval));
for h in hlist do
  f_approx := evalf( (−f(x0 − h) + 4*f(x0 + h) − 3*f(x0 + 2*h) − f(x0)) / (2*h^2) ) :
  print(cat("h = ", h, ", Approximation f''(1) = ", f_approx));
end do:
# (c) Error bounds (truncation term)
f4 := diff(f(x), x$4) :
maxval := 0 :
for i from 80 to 120 do
  xi := i/100 :
  val := abs(evalf(subs(x = xi, f4))) :
  if val > maxval then maxval := val : end if:
end do:
print("-----------------------------------------------------");
print("(c) Error bounds:");
for h in hlist do
  ErrorBound := evalf((h^2/12)*maxval) :
  print(cat("h = ", h, ", Upper bound ≈ ", ErrorBound));
end do:
# (d) Absolute errors
print("-----------------------------------------------------");
print("(d) Absolute errors:");
for h in hlist do
  f_approx := evalf( (−f(x0 − h) + 4*f(x0 + h) − 3*f(x0 + 2*h) − f(x0)) / (2*h^2) ) :
  abs_err := abs(f_approx − trueval) :
  print(cat("h = ", h, ", Absolute error = ", abs_err));
end do:
print("-----------------------------------------------------");
```

```
"----------------------------------------------------"
"(a) Finite difference formula for f'(x0):"
"Expanded formula:"
"f'(x0) ≈ [-f(x0 - h) + 4*f(x0 + h) - 3*f(x0 + 2*h) - f(x0)] / (2*h^2)"
trueval := 4.992923170
"----------------------------------------------------"
"(b) Numerical approximations for f'(1):"
"True f'(1) = 4.992923170"
"h = .1, Approximation f'(1) = -95.35227990"
"h = .5e-1, Approximation f'(1) = -321.7075314"
"h = .1e-1, Approximation f'(1) = -7218.093285"
"----------------------------------------------------"
"(c) Error bounds:"
"h = .1, Upper bound ≈ .1855613723e-1"
"h = .5e-1, Upper bound ≈ .4639034308e-2"
"h = .1e-1, Upper bound ≈ .1855613723e-3"
"----------------------------------------------------"
"(d) Absolute errors:"
"h = .1, Absolute error = 100.3452031"
"h = .5e-1, Absolute error = 326.7004546"
"h = .1e-1, Absolute error = 7223.086208"
"----------------------------------------------------"
```

a) f''(x0)= (-f(x0 - h)+f(x0+h)-3f(x0+2h)-f(x0))/2h^2

b) h (.1) -> 4.9770
   h(.005) ->4.9930
   h(.001) ->4.9929

c) h (.1) -> .0186
   h(.005) ->.00464
   h(.001) ->.001856

d) h (.1) -> .0159
   h(.005) ->.0010
   h(.001) ->.00000

Problem 2

```
g := x -> x*sin(x) :
x0 := 1.5;
# Define h values to test
hlist := [0.1, 0.05, 0.01, 0.005, 0.001] :
# Forward, 3-point, 5-point formulas
printf("Approximating g'(1.5) using different finite differences:\n\n");
for h in hlist do
  forward := evalf((g(x0 + h) - g(x0))/h);
  threept := evalf((g(x0 + h) - g(x0 - h))/(2*h));
  fivept  := evalf((-g(x0 + 2*h) + 8*g(x0 + h) - 8*g(x0 - h) + g(x0 - 2*h))/(12*h));
  printf("h = %.4f  Forward: %.10f  3-pt: %.10f  5-pt: %.10f\n",
      h, forward, threept, fivept);
end do;
# True derivative for comparison
gprime := diff(g(x), x) :
trueval := evalf(subs(x = x0, gprime));
printf("\nTrue derivative g'(1.5) = %.10f\n", trueval);
# Optional: Absolute errors
printf("\nAbsolute errors compared to true value:\n");
for h in hlist do
  forward := evalf((g(x0 + h) - g(x0))/h);
  threept := evalf((g(x0 + h) - g(x0 - h))/(2*h));
  fivept  := evalf((-g(x0 + 2*h) + 8*g(x0 + h) - 8*g(x0 - h) + g(x0 - 2*h))/(12*h));
  f_err := abs(forward - trueval);
  t_err := abs(threept - trueval);
  p_err := abs(fivept - trueval);
  printf("h = %.4f  Forward: %.2e  3-pt: %.2e  5-pt: %.2e\n",
      h, f_err, t_err, p_err);
end do;
```

$x0 := 1.5$

Approximating g'(1.5) using different finite differences:

$forward := 1.030752850$
$threept := 1.098440715$
$fivept := 1.103583834$

h = 0.1000 Forward: 1.0307528500 3-pt: 1.0984407150 5-pt: 1.1035838340

$forward := 1.068447100$
$threept := 1.102309980$
$fivept := 1.103599737$

h = 0.0500 Forward: 1.0684471000 3-pt: 1.1023099800 5-pt: 1.1035997370

$forward := 1.096775300$
$threept := 1.103549150$
$fivept := 1.103600750$

h = 0.0100 Forward: 1.0967753000 3-pt: 1.1035491500 5-pt: 1.1036007500

$forward := 1.100201000$
$threept := 1.103587900$
$fivept := 1.103600833$

h = 0.0050 Forward: 1.1002010000 3-pt: 1.1035879000 5-pt: 1.1036008330

$forward := 1.102923000$
$threept := 1.103600500$
$fivept := 1.103600250$

h = 0.0010 Forward: 1.1029230000 3-pt: 1.1036005000 5-pt: 1.1036002500

$trueval := 1.103600789$

True derivative g'(1.5) = 1.1036007890

Absolute errors compared to true value:

$forward := 1.030752850$
$threept := 1.098440715$
$fivept := 1.103583834$
$f\_err := 0.072847939$
$t\_err := 0.005160074$
$p\_err := 0.000016955$

h = 0.1000 Forward: 7.28e-02 3-pt: 5.16e-03 5-pt: 1.70e-05

$forward := 1.068447100$
$threept := 1.102309980$
$fivept := 1.103599737$
$f\_err := 0.035153689$
$t\_err := 0.001290809$
$p\_err := 1.052 \times 10^{-6}$

h = 0.0500 Forward: 3.52e-02 3-pt: 1.29e-03 5-pt: 1.05e-06

$forward := 1.096775300$
$threept := 1.103549150$
$fivept := 1.103600750$
$f\_err := 0.006825489$
$t\_err := 0.000051639$
$p\_err := 3.9 \times 10^{-8}$

h = 0.0100 Forward: 6.83e-03 3-pt: 5.16e-05 5-pt: 3.90e-08

$forward := 1.100201000$
$threept := 1.103587900$
$fivept := 1.103600833$
$f\_err := 0.003399789$
$t\_err := 0.000012889$
$p\_err := 4.4 \times 10^{-8}$

h = 0.0050 Forward: 3.40e-03 3-pt: 1.29e-05 5-pt: 4.40e-08

$forward := 1.102923000$
$threept := 1.103600500$
$fivept := 1.103600250$
$f\_err := 0.000677789$
$t\_err := 2.89 \times 10^{-7}$
$p\_err := 5.39 \times 10^{-7}$

h = 0.0010 Forward: 6.78e-04 3-pt: 2.89e-07 5-pt: 5.39e-07

=

Problem 3

```
> # function f(x)
  f := x -> x^2*ln(x + 3) + cos(x)*exp(-3*x^2) :
  x0 := 1 : # point where derivative is evaluated
  h := 0.01 : # step size
  # Compute the approximation
  f_approx := evalf( (f(x0 + h) - f(x0 - 3*h)) / (4*h)) :
  # Compute the true derivative
  fprime := diff(f(x), x) :
  trueval := evalf(subs(x=x0, fprime)) :
  # Compute absolute error
  abs_err := evalf(abs(f_approx - trueval)) :
  # Print results
  printf("Approximation f'(%.2f) = %.10f\n", x0, f_approx) :
  printf("True derivative f'(%.2f) = %.10f\n", x0, trueval) :
  printf("Absolute error = %.8e\n", abs_err) :

Approximation f'(1.00) = 2.7687703000
True derivative f'(1.00) = 2.8192939420
Absolute error = 5.05236420e-02
```

Problem 4

```
> f := t -> cos(Pi*t)*3^(1 - t^2) :
  a := -1 : b := 1 : tol := 1e-5 :
  # Trapezoidal Rule
  Iprev := 0 :
  for n from 2 by 2 to 10000 do
    h := evalf( (b - a) / n) :
    Itrap := evalf( (h/2)*(f(a) + 2*add(f(a + i*h), i = 1 ..n-1) + f(b))) :
    if n > 2 and abs(Itrap - Iprev) < tol then
      break
    end if:
    Iprev := Itrap :
  end do:
  printf("Trapezoidal: I = %.8f, h = %.8f, n = %d\n", Itrap, h, n) :
  # Simpson's Rule
  Iprev2 := 0 :
  for n from 2 by 2 to 10000 do
    h := evalf( (b - a) / n) :
    Isimp := evalf( (h/3)*(f(a) + 4*add(f(a + i*h), i = 1 ..n-1, 2)
             + 2*add(f(a + i*h), i = 2 ..n-2, 2) + f(b))) :
    if n > 2 and abs(Isimp - Iprev2) < tol then
      break
    end if:
    Iprev2 := Isimp :
  end do:
  printf("Simpson's:  I = %.8f, h = %.8f, n = %d\n", Isimp, h, n) :

Trapezoidal: I = 0.89507855, h = 0.02325581, n = 86
Simpson's:  I = 0.89486025, h = 0.07142857, n = 28
```

Problem5

```
# data
z := [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12] :
d := [0, 2.6, 3.2, 4.8, 5.6, 6, 6.2, 6, 5.6, 4.8, 3.2, 2.6, 0] :
# radii
r := [seq(d[i]/2, i = 1 ..nops(d))] :
# Simpson's composite (n = 12 subintervals, h = 1)
h := 1.0 :
n := nops(r) - 1 :      # should be 12
# Simpson: (h/3)[f0 + fn + 4*sum odd + 2*sum even]
sum_odd := add(r[i], i = 2 ..n, 2) :   # indices 2,4,... in Maple indexing of r list
sum_even := add(r[i], i = 3 ..n - 1, 2) : # indices 3,5,...
vals := r :
I_r := (h/3) * (vals[1] + vals[n + 1] + 4 * add(vals[i], i = 2 ..n, 2) + 2 * add(vals[i], i = 3 ..n - 1, 2));
# For r^2
vals2 := [seq(vals[i]^2, i = 1 ..n + 1)] :
I_r2 := (h/3) * (vals2[1] + vals2[n + 1] + 4 * add(vals2[i], i = 2 ..n, 2) + 2 * add(vals2[i], i = 3 ..n - 1, 2));
# Surface area and volume
SA := evalf(2 * Pi * I_r) :
V := evalf(Pi * I_r2) :
printf("Integral of r dz ≈ %.6f\n", I_r);
printf("Integral of r^2 dz ≈ %.6f\n", I_r2);
printf("Surface area SA ≈ %.6f in^2\n", SA);
printf("Volume V      ≈ %.6f in^3\n", V);
```

$$I\_r := 25.80000000$$

$$I\_r2 := 64.14000000$$

```
Integral of r dz &approx; 25.800000
Integral of r^2 dz &approx; 64.140000
Surface area SA &approx; 162.106181 in^2
Volume V       &approx; 201.501753 in^3
```

Problem6

```
> f := x -> sin(x) * exp(-x^2) :
  # Integration limits
  a := -2 :
  b := 0 :

  nodes := table([
   2 = [-1/sqrt(3), 1/sqrt(3)],
   3 = [0, -sqrt(3/5), sqrt(3/5)],
   4 = [-sqrt((3/7) - (2/7)*sqrt(6/5)), sqrt((3/7) - (2/7)*sqrt(6/5)),
       -sqrt((3/7) + (2/7)*sqrt(6/5)), sqrt((3/7) + (2/7)*sqrt(6/5))]
  ]) :
  weights := table([
   2 = [1, 1],
   3 = [8/9, 5/9, 5/9],
   4 = [(18 + sqrt(30))/36, (18 + sqrt(30))/36, (18 - sqrt(30))/36, (18 - sqrt(30))/36]
  ]) :
  Gauss := proc(n)
   local xi, wi, i, sum, x, w, t, result;
   xi := nodes[n]; wi := weights[n];
   sum := 0;
   for i to n do
    # Map node from [-1,1] to [a,b]
    t := (b - a)/2 * xi[i] + (b + a)/2;
    sum := sum + wi[i]*f(t);
   od;
   result := (b - a)/2 * sum;
   return evalf(result);
   end proc:
  # Compute for n = 2, 3, 4
  I2 := Gauss(2);
  I3 := Gauss(3);
  I4 := Gauss(4);

  # Display results
  printf("n=2: %.10f\nn=3: %.10f\nn=4: %.10f\n", I2, I3, I4);
```

$$I2 := -0.4261499406$$
$$I3 := -0.4165126830$$
$$I4 := -0.4217209782$$

```
n=2: -0.4261499406
n=3: -0.4165126830
n=4: -0.4217209782
```

Problem 7

```
# Define unknowns
a := 'a': b := 'b': c := 'c': d := 'd': e := 'e':
 # system of equations
eqns := {
  a + b + c = 2,              # integral of f(x) = 1
  -a + c + d + e = 0,         # integral of f(x) = x
  a + c - 2*d + 2*e = 2/3,    # integral of f(x) = x^2
  -a + c + 3*d + 3*e = 0,     # integral of f(x) = x^3
  a + c - 4*d + 4*e = 2/5     # integral of f(x) = x^4
 };

sol := solve(eqns, {a, b, c, d, e});
 sol;
printf("Quadrature formula for ∫_{-1}^{1} f(x) dx:\n");
printf("∫_{-1}^{1} f(x) dx ≈ %a*f(-1) + %a*f(0) + %a*f(1) + %a*f'(-1) + %a*f'(1)\n",
    sol[a], sol[b], sol[c], sol[d], sol[e]);
```

$$eqns := \left\{a + b + c = 2, -a + c + d + e = 0, -a + c + 3\,d + 3\,e = 0, a + c - 4\,d + 4\,e = \frac{2}{5}, a + c - 2\,d + 2\,e = \frac{2}{3}\right\}$$

$$sol := \left\{a = \frac{7}{15}, b = \frac{16}{15}, c = \frac{7}{15}, d = \frac{1}{15}, e = -\frac{1}{15}\right\}$$

$$\left\{a = \frac{7}{15}, b = \frac{16}{15}, c = \frac{7}{15}, d = \frac{1}{15}, e = -\frac{1}{15}\right\}$$

```
Quadrature formula for &int;_{-1}^{1} f(x) dx:
&int;_{-1}^{1} f(x) dx &approx; {a = 7/15, b = 16/15, c = 7/15, d = 1/15, e = -1/15}[a]*f(-1) + {a = 7/15, b = 16/15, c = 7/15, d = 1/15, e = -1/15}[b]*f
(0) + {a = 7/15, b = 16/15, c = 7/15, d = 1/15, e = -1/15}[c]*f(1) + {a = 7/15, b = 16/15, c = 7/15, d = 1/15, e = -1/15}[d]*f'(-1) + {a = 7/15, b = 16/15,
c = 7/15, d = 1/15, e = -1/15}[e]*f'(1)
```

$$\int_{-1}^{1} f(x)dx = 7/15f(-1)+16/15f(0)+7/15f(1)+1/15f'(-1)-1/15f'(1)$$